Product Catalog REST API — 5-Endpoint CRUD

Beginner-friendly Spring Boot example using an in-memory **ArrayList** (no database). Comments in the code explain what each statement does.

Endpoints included: get all products, get one product, create a product, update a product, and delete a product.

1. Product.java — Model class

This class represents one product. Spring/Jackson can use its fields and getters/setters to convert between Java objects and JSON.

```
package com.example.demo;

public class Product {

    // Unique ID used to identify each product
    private int id;

    // Product name, for example "Keyboard"
    private String name;

    // Price of the product
    private double price;

    // Empty constructor used by frameworks when needed
    public Product() {
    }

    // Constructor used to create a product with values
    public Product(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    // Returns the product ID
    public int getId() {
        return id;
    }

    // Sets the product ID
    public void setId(int id) {
        this.id = id;
    }

    // Returns the product name
    public String getName() {
        return name;
    }

    // Sets the product name
    public void setName(String name) {
        this.name = name;
    }

    // Returns the product price
    public double getPrice() {
        return price;
    }

    // Sets the product price
    public void setPrice(double price) {
        this.price = price;
    }
}
```

2. ProductController.java — REST API

This controller stores products in an ArrayList and exposes five HTTP endpoints. The list is in memory, so data resets when the application restarts.

```
package com.example.demo;

import java.util.ArrayList;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController  // Makes this class handle REST requests and return data as JSON
@RequestMapping("/products")  // Base URL for every endpoint in this controller
```

```
public class ProductController {

    // In-memory list: no database is used
    private List<Product> products = new ArrayList<>();

    // Constructor runs when Spring creates this controller
    public ProductController() {
        // Add two sample products so GET requests return data immediately
        products.add(new Product(1, "Keyboard", 1200.0));
        products.add(new Product(2, "Mouse", 600.0));
    }

    // 1. GET /products
    // Returns the complete product list
    @GetMapping
    public ResponseEntity<List<Product>> getAllProducts() {
        return ResponseEntity.ok(products);  // HTTP 200 OK
    }

    // 2. GET /products/{id}
    // Returns one product matching the ID in the URL
    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable int id) {
        // Search the list for a product with the requested ID
        for (Product product : products) {
            if (product.getId() == id) {
                return ResponseEntity.ok(product);  // Found: HTTP 200
            }
        }

        // No matching product was found: HTTP 404 Not Found
        return ResponseEntity.notFound().build();
    }

    // 3. POST /products
    // Creates a new product using the JSON request body
    @PostMapping
    public ResponseEntity<Product> addProduct(@RequestBody Product product) {
        // Reject a product if its ID is already in use
        for (Product existing : products) {
            if (existing.getId() == product.getId()) {
                return ResponseEntity.status(HttpStatus.CONFLICT).build(); // HTTP 409
            }
        }

        // Add the new product to the in-memory list
        products.add(product);

        // Return the created product with HTTP 201 Created
        return ResponseEntity.status(HttpStatus.CREATED).body(product);
    }

    // 4. PUT /products/{id}
    // Replaces the product details for the given ID
    @PutMapping("/{id}")
    public ResponseEntity<Product> updateProduct(
            @PathVariable int id,
            @RequestBody Product updatedProduct) {

        // Find the product that needs to be updated
        for (int i = 0; i < products.size(); i++) {
            if (products.get(i).getId() == id) {

                // Keep the URL ID as the product ID, even if JSON has another ID
                updatedProduct.setId(id);

                // Replace the old product with the updated product
                products.set(i, updatedProduct);

                // Return updated details with HTTP 200 OK
                return ResponseEntity.ok(updatedProduct);
            }
        }

        // Cannot update a product that does not exist
        return ResponseEntity.notFound().build(); // HTTP 404
    }

    // 5. DELETE /products/{id}
    // Deletes a product using its ID
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable int id) {
        // Search for the product to delete
        for (int i = 0; i < products.size(); i++) {
            if (products.get(i).getId() == id) {
```

```
                // Remove the matching product from the list
                products.remove(i);

                // Deletion succeeded: HTTP 204 No Content
                return ResponseEntity.noContent().build();
            }
        }

        // No product with this ID exists
        return ResponseEntity.notFound().build(); // HTTP 404
    }
}
```

3. Main application class

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication  // Enables Spring Boot auto-configuration and component scanning
public class ProductCatalogApplication {

    public static void main(String[] args) {
        // Starts the embedded server and the Spring application
        SpringApplication.run(ProductCatalogApplication.class, args);
    }
}
```

4. Endpoint summary

GET /products — list all products (200 OK)
GET /products/1 — get product with ID 1 (200 OK, or 404 Not Found)
POST /products — create a product (201 Created; 409 Conflict if ID already exists)
PUT /products/1 — update product with ID 1 (200 OK, or 404 Not Found)
DELETE /products/1 — delete product with ID 1 (204 No Content, or 404 Not Found)

5. Example JSON for POST and PUT

```
{
  "id": 3,
  "name": "Monitor",
  "price": 8500.0
}
```

6. How to test

1. Create a Spring Boot project with the **Spring Web** dependency.
2. Put these three Java files in the same package (com.example.demo).
3. Run **ProductCatalogApplication**.
4. Use Postman, Insomnia, or curl to send requests to **http://localhost:8080/products**.
5. For POST and PUT, choose raw JSON as the request body and set **Content-Type: application/json**.

7. Important notes

• **@RestController**: Handles REST requests and serializes returned Java objects as JSON.
• **@GetMapping**, **@PostMapping**, **@PutMapping**, **@DeleteMapping**: Map HTTP methods to Java methods.
• **@PathVariable**: Reads the ID from the URL.
• **@RequestBody**: Converts incoming JSON into a Product object.
• **ResponseEntity**: Lets the method choose an HTTP status code and response body.
• **ArrayList**: Stores products only while the application is running; restarting clears changes and recreates the two sample products.